

Job Parameters

Task Parameters

Task Values

dynamic value reference

Task Values

=====

if a task want to set a parameter, so that other task can retrieve that dynamically.

```
dbutils.jobs.taskValues.set(key="has_duplicates", value=duplicate_exists)
```

```
dbutils.jobs.taskValues.get(taskKey = "orders_silver_cleaned",  
key="has_duplicates")
```

task1 -> task2

task1

```
dbutils.jobs.taskValues.set(  
key = "catalog_name"  
value = "databrickslearning"  
)
```

task2

```
dbutils.jobs.taskValues.get(  
taskKey = "task1",  
key = "catalog_name"  
)
```

task values represent a more advanced pattern than parameters, they are computed at runtime and enable dynamic communication between tasks.

-- parameters are set before execution, task values are created during execution.

Dynamic Value Reference

=====

allow referencing values at runtime from job and task context..

{{ }}

{{job.start_time.day}}

{{task.name}}

{{tasks.<taskname>.values.<myvalue>}}

{{tasks.orders_silver_cleaned.values.has_duplicates}}

for accessing output from an upstream task

- job parameters
- task parameters
- task values
- dynamic value reference

=====

job can be executed on different compute

- interactive cluster
- job cluster
- serverless
- SQL warehouse

each task within a job can have its own compute

tasks in the same job either:

- share the same compute
- use different computes as required.

performance optimized (off)

- focus on cost efficiency
- longer startup times (typically 4-6 minutes)
- best for non urgent workloads

performance optimized (on)

- faster job execution and startup
- ideal for time sensitive workloads
- apply for serverless compute

common workload patterns

- sequential (bronze -> silver -> gold)
- funnel (multiple data sources, data collections)

task 1

task 2 -> task 4

task 3

- fan-out (single data source, data distribution)

task 2

task 1

task 3

common task configuration options

- task parameters
- dynamic value reference
- notification alerts
- retries

Retry policy

=====

consider factors like

- failure types: transient network issues might require immediate retries, while data quality issues might not.
- resource impact: retrying resource intensive tasks too aggressively can cause cluster resource contention.
- downstream dependencies: failed tasks might impact other workflows, making retry interval critical.

Triggers

=====

A trigger is a rule that starts a job run based on specific condition or schedule.

1. time based schedule
2. continuous trigger (always on)
3. file arrival event
4. manual trigger
5. table update

=> scheduled triggers

cron expression for sophisticated schedules like "every 15 minutes during business hours" or "first monday of each month"

use cases:

daily etl: process yesterdays data every morning at 6 am.

weekly reports: monday morning.

=> file arrival triggers - automatically triggers job when new file are detected in a specified storage location

use cases: partner data feed

=> continuous trigger - runs a new job as soon as previous one finishes.

use cases

- fraud detection

=> Manual trigger - let jobs run on demand.

best for adhoc runs, debugging, one off backfills

- backfill processing
- emergency processing
- testing and validation

=> Table update triggers: automatically starts a job when specified tables are updated

For each task

=====

- the for each task - the top level container task that manages the loop
- a nested task - the actual task that executes each iteration

use case

- process data for each state/region in parallel
- time period processing: handle different data ranges with same logic.

best practises

=====

- use job or serverless cluster in production
- use service principal for job ownership
- leverage the repair and run for reducign cost of re processing.
- parameterize tasks for reusability and flexibility